

UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

Facultatea de Automatică și Calculatoare

Proiectarea Sistemelor Numerice

Joc Mini-Sudoku pe Placă FPGA

Implementare VHDL cu afișaj VGA și SSD

Platformă: Nexys 4 (Xilinx Artix-7 XC7A100T)

Limbaj: VHDL | Mediu: Vivado

2024 – 2025

Cuprins

Capitolul 1: Specificații și Cerința Proiectului

Capitolul 2: Arhitectura Sistemului

2.1. Black Box – Vedere de ansamblu

2.2. Componente principale

2.3. Diagrama bloc detaliată

Capitolul 3: Descrierea Componentelor VHDL

3.1. Pachetul sudoku_pkg

3.2. Modulul top_vga_div (Top Level)

3.3. Modulul logica_jocului

3.4. Modulul vga

3.5. Modulul afisaj_ssd

3.6. Modulul debouncer

3.7. Modulul div_frecventa

Capitolul 4: Logica VGA

4.1. Timing VGA 640×480

4.2. Randarea grilei

4.3. Codul culorilor

Capitolul 5: Manual de Utilizare

Capitolul 6: Justificarea Soluțiilor de Design

Capitolul 7: Posibilități de Dezvoltare

Capitolul 8: Bibliografie

Capitolul 1: Specificații și Cerința Proiectului

1.1. Descrierea generală

Proiectul **Mini-Sudoku FPGA** constă în implementarea unui joc de tip Sudoku simplificat (grilă 4×4) pe o placă de dezvoltare **Nexys 4** cu FPGA Xilinx Artix-7. Jocul este complet descris în limbajul **VHDL** și utilizează două metode de afișaj: un monitor VGA (640×480 px) pentru grila de joc și afișorul cu 7 segmente (SSD) pentru indicarea stării finale (câștig sau eșec).

1.2. Reguli Sudoku 4×4

Varianta 4×4 a jocului Sudoku urmează aceleași principii ca versiunea standard, dar simplificată:

- Grila este formată din 4 linii × 4 coloane (16 celule).
- Fiecare linie trebuie să conțină exact câte o apariție a cifrelor 1, 2, 3 și 4.
- Fiecare coloană trebuie să conțină exact câte o apariție a cifrelor 1, 2, 3 și 4.
- Fiecare dintre cele 4 cadrane 2×2 trebuie să conțină toate cele 4 cifre.
- Celulele inițiale (pre-completate) nu pot fi modificate de jucător.

În această implementare, grila inițială este complet goală iar jucătorul trebuie să completeze toate cele 16 celule. Soluția unică acceptată este:

1	2	3	4
3	4	1	2
2	1	4	3
4	3	2	1

Tabel 1.1 – Soluția unică acceptată (1=Roșu, 2=Verde, 3=Albastru, 4=Galben)

1.3. Resurse hardware utilizate

Semnal	Descriere
CLK (E3)	Ceas de sistem 100 MHz – intrare principală
RST (C12)	Buton de reset asincron (centru)
BTN_UP (M18)	Buton navigare cursor sus
BTN_DOWN (P18)	Buton navigare cursor jos
BTN_LEFT (P17)	Buton navigare cursor stânga

BTN_RIGHT (M17)	Buton navigare cursor dreapta
BTN_ENTER (N17)	Buton confirmare valoare
SW[0..4]	Switch-uri pentru selectarea valorii (1-4)
VGA (HS/VS/RGB)	leșire video la monitor 640×480 @ 60Hz
SSD (AN/CAT)	Afișor 7 segmente × 8 cifre – stare joc

Capitolul 2: Arhitectura Sistemului

2.1. Black Box – Vedere de Ansamblu

Prima etapă de proiectare constă în identificarea **intrărilor și ieșirilor** sistemului, privit ca o cutie neagră, fără a ști ce se află în interior. Această abordare *top-down* permite stabilirea clară a interfeței cu utilizatorul și cu placa FPGA.

Intrări:

- CLK – ceas sistem (100 MHz, Nexys 4)
- RST – reset global
- 5 butoane direcționale + enter
- SW[4:0] – selectare valoare de introdus

Ieșiri:

- HS, VS – semnale sincronizare VGA
- R[3:0], G[3:0], B[3:0] – culori pixel VGA (4 biți/canal)
- AN[7:0] – activare anodizi SSD
- CAT[6:0] – catodizi (segmente) SSD

2.2. Componente principale

Sistemul este descompus în 5 module VHDL interconectate prin fișierul top-level:

Semnal	Descriere
<code>div_frecventa</code>	Divide ceasul de la 100 MHz la 25 MHz (pixel clock VGA)
<code>debouncer</code>	Filtrează zgomotul mecanic al butoanelor (×5 instanțe)
<code>logica_jocului</code>	FSM: gestionează grila 4×4, cursorul, introducerea valorilor, detectarea câștigului/eșecului
<code>vga</code>	Generează semnalele VGA și randează grila colorată pe monitor
<code>afisaj_ssd</code>	Afișează mesajul WIN sau LOSE pe afișorul cu 7 segmente

2.3. Diagrama Bloc Detaliată

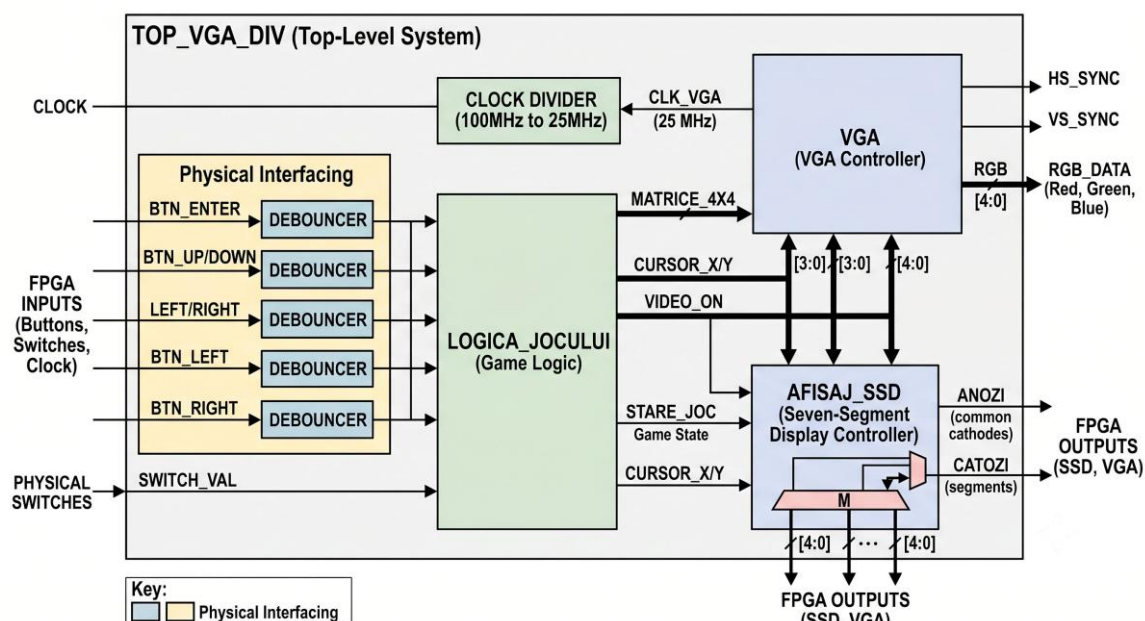


Figura 2.1 – Diagrama bloc a sistemului Mini-Sudoku FPGA

Semnalele interne cheie care leagă modulele sunt: **clk_25MHz** (ceasul divizat, distribuit tuturor modulelor), **tabla_semnal** (matricea 4×4 de tip *matrice_joc*), **cx_semnal/cy_semnal** (poziția cursorului), și **stadiu_joc_semnal** (0=în joc, 1=câștig, 2=eșec).

2.4. Diagrama de Stări

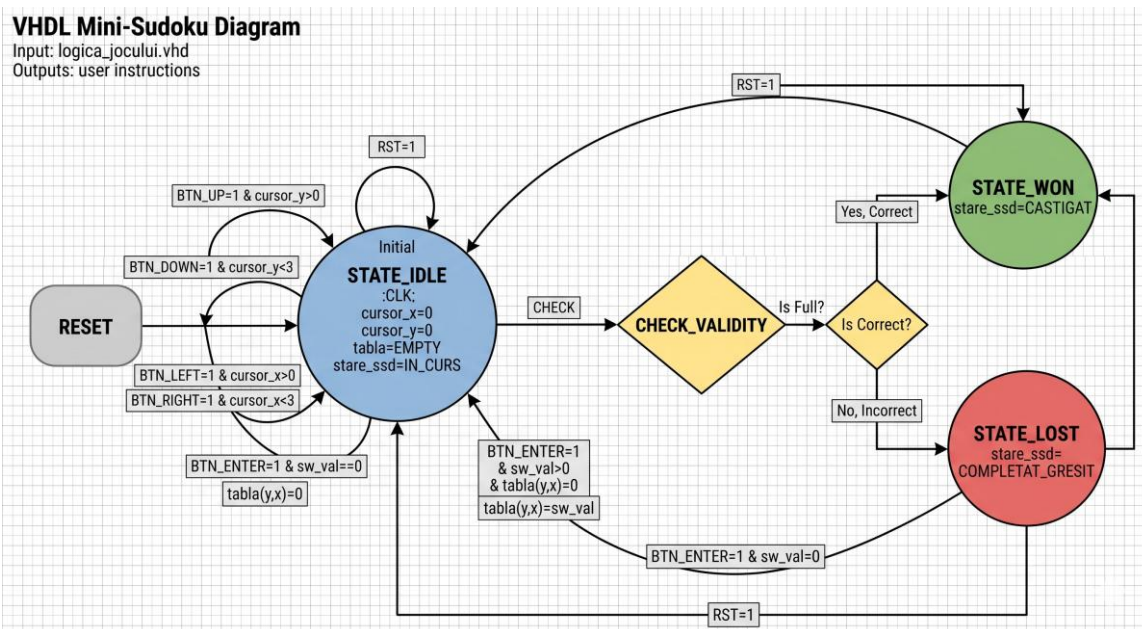


Figura 2.2 – Diagrama de stări a logicii jocului

Jocul funcționează ca un automat cu 3 stări implicite:

- IDLE – tabla este goală (toate celulele = 0), jucătorul poate naviga cursorul și introduce valori

- CÂȘTIG – tabla este completă și corespunde soluției corecte; SSD afișează "WIN"
- EȘEC – tabla este completă dar incorectă; SSD afișează "LOSE"

Capitolul 3: Descrierea Componentelor VHDL

3.1. Pachetul sudoku_pkg

Pachetul **sudoku_pkg** definește tipul de date personalizat **matrice_joc**, care reprezintă grila 4×4. Acesta este inclus în toate modulele care necesită accesul la grilă.

```
package sudoku_pkg is
    type matrice_joc is array (0 to 3, 0 to 3) of integer range 0 to 4;
end package;
```

Tipul **matrice_joc** este un array 2D de 4×4 elemente, fiecare element putând lua valori întregi de la 0 la 4. Valoarea 0 semnifică o celulă goală, iar valorile 1–4 reprezintă cifrele introduse de jucător.

3.2. Modulul top_vga_div (Top Level)

Modulul top-level realizează interconectarea tuturor componentelor. Acesta instanțiază câte o copie din fiecare modul și le leagă prin semnale interne.

Semnale interne:

```
signal clk_25MHz      : std_logic := '0';
signal db_up, db_down,
       db_left, db_right,
       db_enter       : std_logic;
signal cx_semnal,
       cy_semnal      : integer range 0 to 3;
signal tabla_semnal   : matrice_joc;
signal stadiu_joc_semnal : integer range 0 to 2;
```

Instanțierea modulelor:

```
map_div:    div_frecventa  port map (rst=>rst_t, clk=>clk_t,
                                       clk_divizat=>clk_25MHz);

deb_up:     debouncer      port map (clk=>clk_25MHz,
                                       btn_in=>btn_up_t, btn_out=>db_up);
-- ... (similar pentru celelalte 4 butoane) ...

map_logica: logica_jocului port map (
    clk=>clk_25MHz, rst=>rst_t,
    btn_sus=>db_up, btn_jos=>db_down,
    btn_stg=>db_left, btn_dr=>db_right,
    btn_enter=>db_enter, sw=>sw_t,
    cursor_x_out=>cx_semnal, cursor_y_out=>cy_semnal,
    tabla_out=>tabla_semnal,
    stare_joc=>stadiu_joc_semnal);

map_vga:    vga            port map (
    rst=>rst_t, clk=>clk_25MHz,
    tabla_in=>tabla_semnal,
```

```

cursor_x=>cx_semnal, cursor_y=>cy_semnal,
hs=>hs_t, vs=>vs_t,
r=>r_t, g=>g_t, b=>b_t);

```

3.3. Modulul logica_jocului

Acesta este **creierul jocului**. Gestionează: mișcarea cursorului pe grilă, introducerea valorilor prin switch-uri, și detectarea stării finale.

Interfața modulului:

Semnal	Descriere
clk, rst	Ceas 25 MHz și reset asincron
btn_sus/jos/stg/dr	Semnale de navigare cursor (debounced)
btn_enter	Confirmare introducere valoare
sw[4:0]	Switch-uri: sw(4)→4, sw(3)→3, sw(2)→2, sw(1)→1
cursor_x_out, cursor_y_out	Poziția curentă a cursorului (0..3)
tabla_out	Matricea 4×4 curentă
stare_joc	0=în joc, 1=câștig, 2=eșec

Logica de navigare a cursorului:

```

process(clk)
begin
    if rising_edge(clk) then
        if rst = '1' then
            cx <= 0; cy <= 0;
            tabla <= ((0,0,0,0), (0,0,0,0), (0,0,0,0), (0,0,0,0));
        else
            if btn_sus = '1' then
                if cy > 0 then cy <= cy - 1; end if;
            end if;
            if btn_jos = '1' then
                if cy < 3 then cy <= cy + 1; end if;
            end if;
            -- similar pentru stanga/dreapta
            if btn_enter = '1' then
                if tabla(cy, cx) = 0 or valoare_sw = 0 then
                    tabla(cy, cx) <= valoare_sw;
                end if;
            end if;
        end if;
    end if;
end process;

```

O celulă poate fi **modificată** numai dacă este goală (= 0) sau dacă jucătorul dorește să o șteargă (valoare_sw = 0). Aceasta protejează celulele deja completate. Dacă sw-urile sunt toate 0, valoarea introdusă este 0 (ștergere).

Detectarea câștigului/eșecului:

```
constant WIN_BOARD : matrice_joc := (  
    (1, 2, 3, 4),  
    (3, 4, 1, 2),  
    (2, 1, 4, 3),  
    (4, 3, 2, 1)  
);  
  
process(tabla)  
    variable is_full      : boolean;  
    variable is_correct   : boolean;  
begin  
    is_full := true; is_correct := true;  
    for i in 0 to 3 loop  
        for j in 0 to 3 loop  
            if tabla(i,j) = 0 then is_full := false; end if;  
            if tabla(i,j) /= WIN_BOARD(i,j) then  
                is_correct := false; end if;  
        end loop;  
    end loop;  
    if is_full then  
        if is_correct then stare_joc <= 1; -- WIN  
        else stare_joc <= 2; -- LOSE  
        end if;  
    else stare_joc <= 0; -- in joc  
    end if;  
end process;
```

3.4. Modulul vga

Modulul vga generează semnalele standard VGA 640×480 la 60 Hz și desenează grila de joc pe monitor. Funcționează pe baza unui pixel clock de 25 MHz.

Contoarele de scanare orizontală și verticală:

```
-- Constante VGA 640x480 @ 60Hz  
constant hd : integer := 640; -- active pixels  
constant hfp : integer := 16; -- front porch  
constant hsp : integer := 96; -- sync pulse  
constant hbp : integer := 48; -- back porch  
-- Total linie: 800 pixeli  
  
constant vd : integer := 480; -- active lines  
constant vfp : integer := 10;  
constant vsp : integer := 2;  
constant vbp : integer := 33;  
-- Total cadru: 525 linii  
  
process(clk, rst)  
begin  
    if rst='1' then h_cnt<=0; v_cnt<=0;  
    elsif rising_edge(clk) then  
        if h_cnt = h_max then  
            h_cnt <= 0;  
            if v_cnt = v_max then v_cnt <= 0;  
            else v_cnt <= v_cnt + 1; end if;  
        else h_cnt <= h_cnt + 1;  
        end if;  
    end if;  
end process;
```

Generarea culorilor celulelor:

```
-- Culoarea celulei depinde de valoarea din tabla
case tabla_in(rnd_desenat, col_desenata) is
  when 1 => r<=x"F"; g<=x"0"; b<=x"0"; -- Rosu
  when 2 => r<=x"0"; g<=x"F"; b<=x"0"; -- Verde
  when 3 => r<=x"0"; g<=x"0"; b<=x"F"; -- Albastru
  when 4 => r<=x"F"; g<=x"F"; b<=x"0"; -- Galben
  when others => null; -- celula goala ramane neagra/gri
end case;
```

3.5. Modulul afisaj_ssd

Modulul controlează cele 8 afișoare cu 7 segmente ale plăcii Nexys 4. Afișează un mesaj corespunzător stării jocului prin multiplexare rapidă a anodelor.

```
-- Multiplexare: activam cate un SSD la un moment dat
digit_sel <= count(16 downto 14); -- selectia cifrei

-- Mesaj WIN (stare_joc=1): afiseaza "WIN"
when "000" => cat <= "0101011"; -- W
when "001" => cat <= "1111001"; -- I
when "010" => cat <= "1000001"; -- N

-- Mesaj LOSE (stare_joc=2): afiseaza "LOSE"
when "000" => cat <= "0000110"; -- L
when "001" => cat <= "0010010"; -- o
when "010" => cat <= "1000000"; -- S (8-like)
when "011" => cat <= "1000111"; -- E
```

Multiplexarea se realizează la o frecvență suficient de mare (derive din clk_25MHz) astfel încât ochiul uman să perceapă toate cifrele ca și cum ar fi aprinse simultan (frecvență > 60 Hz per SSD).

3.6. Modulul debouncer

Butoanele fizice generate multiple pulsuri (contact bounce) la apăsare. Modulul debouncer elimină acest fenomen prin numărarea stabilității semnalului.

```
-- Butonul trebuie sa fie stabil '1' pentru 0xFFFF
-- cicluri de ceas inainte de a fi acceptat
process(clk)
begin
  if rising_edge(clk) then
    if btn_in = '1' then
      if count = x"FFFF" then
        btn_stare <= '1';
      else
        count <= count + 1;
      end if;
    else
      count <= (others => '0');
      btn_stare <= '0';
    end if;
    btn_stare_veche <= btn_stare;
  end if;
end process;

-- Genereaza un singur puls la front crescator
```

```
btn_out <= '1' when (btn_stare='1' and btn_stare_veche='0')  
           else '0';
```

La 25 MHz, $0xFFFF = 65535$ cicluri = **2.62 ms**. Orice fluctuație sub această durată este ignorată. Ieșirea produce un singur puls de un ciclu de ceas la **frontul crescător** al semnalului debounced.

3.7. Modulul div_frecventa

Placa Nexys 4 furnizează un ceas de 100 MHz. VGA standard $640 \times 480 @ 60\text{Hz}$ necesită un pixel clock de 25 MHz. Modulul div_frecventa realizează această divizare prin numărare.

Principiu: se numără 2 cicluri de ceas la 100 MHz și se face toggle pe semnalul de ieșire
→ rezultă 25 MHz (factor de divizare 4: $100/4 = 25\text{ MHz}$).

Capitolul 4: Logica VGA

4.1. Timing VGA 640×480 @ 60Hz

Standardul VGA 640×480 la 60Hz este unul dintre cele mai răspândite formate video. Generarea semnalului necesită un pixel clock de 25.175 MHz (aproximat la 25 MHz în implementare).

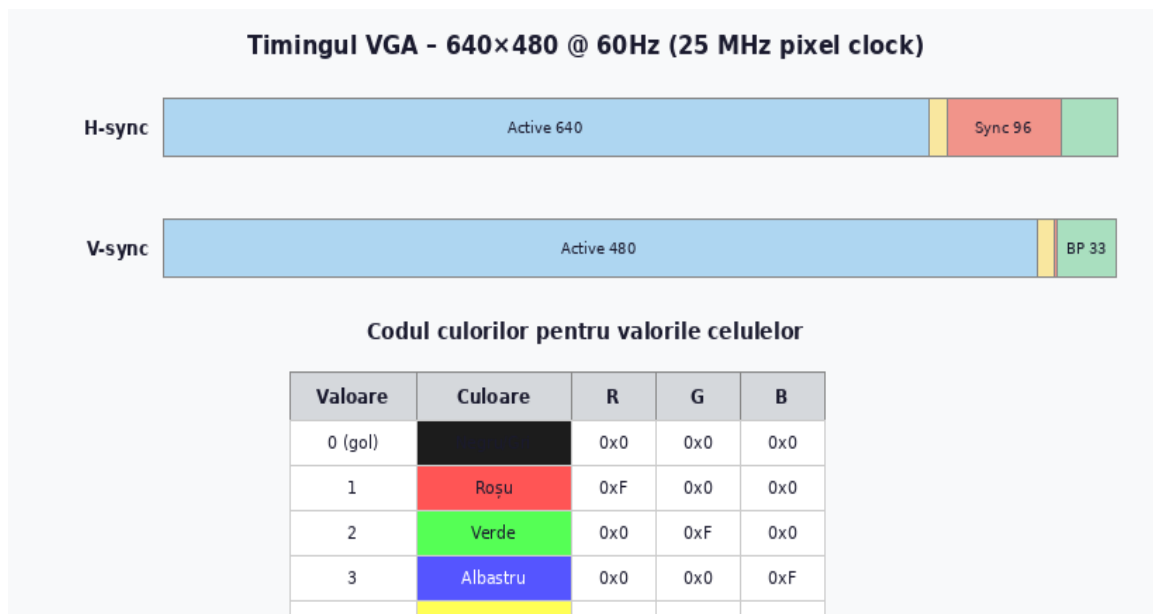


Figura 4.1 – Timingul VGA horizontal/vertical și codul culorilor

Fiecare linie orizontală are 800 pixeli totali (640 activi + 160 porch+sync). Un cadru vertical are 525 linii (480 active + 45 porch+sync). Frecvența de cadru: 25MHz / (800×525) ≈ 59.5 Hz.

4.2. Randarea Grilei

Grila 4×4 este plasată central pe ecran. Dimensiunile și pozițiile celulelor sunt hardcodate în modul:

```
-- Pozitia grilei pe ecran (pixeli)
-- Coloana 0: h=120..220  Coloana 1: h=220..320
-- Coloana 2: h=320..420  Coloana 3: h=420..520
-- Rand 0:   v=40..140    Rand 1:   v=140..240
-- Rand 2:   v=240..340   Rand 3:   v=340..440

-- Liniile grilei (albe) la fiecare 100 pixeli
if ((h_cnt=120 or h_cnt=220 or h_cnt=320 or
    h_cnt=420 or h_cnt=520) and
```

```
(v_cnt>=40 and v_cnt<=440)) or  
(v_cnt=40 or v_cnt=140 or v_cnt=240 or  
v_cnt=340 or v_cnt=440) and  
(h_cnt>=120 and h_cnt<=520)) then  
r<=x"F"; g<=x"F"; b<=x"F"; -- alb
```

Cursorul activ este indicat prin umplerea celulei selectate cu gri închis (#333333) în loc de negru. Valorile introduse sunt afișate ca un pătrat colorat centrat în celulă (30-70% din dimensiunea celulei pe fiecare axă).

4.3. Vizualizarea Jocului pe Monitor

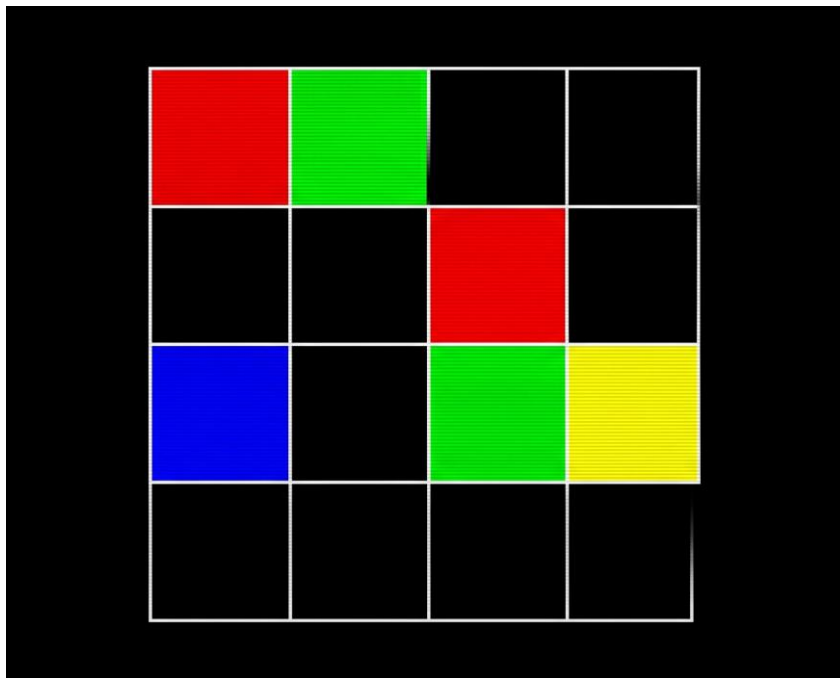


Figura 4.2 – Simulare vizuală a grilei afișate pe monitor (soluția completă)

Fiecare valoare introdusă este reprezentată printr-un pătrat colorat cu margini albe în celula corespunzătoare. Celulele goale rămân negre, iar celula cu cursorul are fundalul gri.

Capitolul 5: Manual de Utilizare

5.1. Configurarea hardware

- Conectați placa Nexys 4 la calculator prin cablu USB-UART.
- Programați placa prin Vivado cu fișierul .bit generat din proiect.
- Conectați un monitor VGA la portul VGA al plăcii Nexys 4.
- Asigurați-vă că monitorul suportă rezoluția 640×480 @ 60Hz.

5.2. Descrierea controalelor

Semnal	Descriere
BTN_UP (sus)	Mută cursorul în sus cu o celulă
BTN_DOWN (jos)	Mută cursorul în jos cu o celulă
BTN_LEFT (stânga)	Mută cursorul la stânga cu o celulă
BTN_RIGHT (dreapta)	Mută cursorul la dreapta cu o celulă
BTN_ENTER (centru)	Confirmă introducerea valorii selectate
SW[0]	Fără efect (valoare 0 = ștergere)
SW[1]	Selectează valoarea 1 (Roșu)
SW[2]	Selectează valoarea 2 (Verde)
SW[3]	Selectează valoarea 3 (Albastru)
SW[4]	Selectează valoarea 4 (Galben)
RST (C12)	Resetează întregul joc (grilă goală, cursor la (0,0))

5.3. Pașii de joc

1. Porniți placa și monitorul. Veți vedea grila 4×4 goală cu cursorul pe celula (0,0) – stânga-sus, indicată prin culoarea gri.
2. Folosiți butoanele direcționale pentru a naviga la celula dorită. Cursorul se deplasează cu un singur pas la fiecare apăsare.
3. Ridicați switch-ul corespunzător valorii dorite (SW[1] pentru 1, SW[2] pentru 2 etc.). Puteți schimba switch-ul oricând înainte de confirmare.
4. Apăsați BTN_ENTER pentru a introduce valoarea în celula curentă. Celula va deveni colorată (roșu/verde/albastru/galben).
5. Repetați pașii 2-4 pentru toate cele 16 celule. O celulă deja completată poate fi ștearsă punând toate switch-urile pe 0 și apăsând ENTER.

6. La completarea ultimei celule, sistemul verifică automat soluția. Dacă este corectă, afișorul SSD va scrie "WIN". Dacă nu, va scrie "LOSE".
7. Apăsați RST oricând pentru a reseta jocul și a începe din nou.

5.4. Sfaturi de joc

- Completați mai întâi liniile/coloanele în care aveți mai multe informații (deducție logică).
- Soluția unică este: linia 1 $\rightarrow$ (1,2,3,4), linia 2 $\rightarrow$ (3,4,1,2), linia 3 $\rightarrow$ (2,1,4,3), linia 4 $\rightarrow$ (4,3,2,1).
- Dacă ați greșit o celulă, navigați la ea, puneți toate switch-urile pe 0 și apăsați ENTER pentru ștergere.

Capitolul 6: Justificarea Soluțiilor de Design

6.1. Alegerea tipului de date matrice_joc

S-a ales definirea unui tip personalizat **array 2D** în loc de un vector plat (*std_logic_vector*) pentru mai multe motive:

- Indexarea bidimensională (tabla(row, col)) este mai intuitivă și mai ușor de citit decât calculul manual al indexului în un vector 1D.
- VHDL suportă nativ array-uri 2D, ceea ce elimină conversia manuală și reduce riscul de erori de indexare.
- Alternativa ar fi fost un vector de 16×3 biți (48 biți total pentru valori 0-4), care ar fi necesitat decodare suplimentară.

6.2. Debouncer prin numărare vs. prin filtrare

S-a ales implementarea debouncer-ului prin **numărare** (contorizarea stabilității semnalului) în loc de un filtru cu registre de deplasare sau RC.

- Numărătoarea de 16 biți la 25 MHz → ~2.6 ms de stabilitate necesară, valoare optimă pentru butoanele Nexys 4.
- Alternativa cu registre de deplasare (shift register debouncer) ar necesita 16-20 flip-flop-uri și logică AND suplimentară – mai costisitoare în resurse FPGA.
- Soluția aleasă este mai ușor de parametrizat (se modifică doar lățimea contorului pentru alt timp de debounce).

6.3. Pixel clock 25 MHz prin divizor vs. PLL

S-a ales **divizarea simplă** a ceasului (100 MHz → 25 MHz) în loc de utilizarea unui PLL (Phase-Locked Loop) al FPGA-ului.

- Divizorul simplu este mai ușor de implementat și de înțeles la nivel educațional.
- PLL-ul ar oferi un semnal mai precis (25.175 MHz exact) și mai curat din punct de vedere spectral, dar implică configurarea unui IP core în Vivado și cunoștințe avansate.
- La rezoluția 640×480, diferența de 0.7% față de frecvența ideală nu produce artefacte vizibile.

6.4. Soluție unică hardcodată vs. puzzle configurabil

Soluția câștigătoare este **hardcodată** ca o constantă VHDL, nu generată aleator sau încărcată din memorie.

- Simplifică considerabil implementarea – nu necesită un generator de numere pseudo-aleatoare sau o memorie ROM cu puzzle-uri.
- Scopul educational al proiectului este demonstrarea conceptelor FPGA (VGA, SSD, FSM), nu complexitatea puzzle-ului.
- Alternativa cu puzzle-uri multiple ar necesita un ROM de cel puțin 16×3 biți $\times$ N puzzle-uri și un mecanism de selectare.

6.5. Afișarea culorilor cu 4 biți per canal

Placa Nexys 4 furnizează **4 biți per canal** (R, G, B) $\rightarrow$ 12 biți total $\rightarrow$ 4096 culori posibile.

- Valorile folosite (0x0 și 0xF) sunt culorile pure la intensitate maximă, ușor de recunoscut vizual.
- O paletă cu mai multe nuanțe ar necesita valori intermediare (ex: 0x8 = 50% intensitate), adăugând complexitate fără beneficiu funcțional clar.

Capitolul 7: Posibilități de Dezvoltare

7.1. Puzzle-uri multiple și generare aleatoare

Cea mai evidentă extensie este adăugarea **mai multor puzzle-uri de start**. Aceasta se poate realiza prin:

- Un ROM intern cu N puzzle-uri (fiecare de 48 biți pentru o matrice 4×4 cu valori 0-4).
- Un generator LFSR (Linear Feedback Shift Register) pentru selectarea aleatoare a unui puzzle la fiecare reset.
- Modificare necesară în logica_jocului: adăugarea unui port de intrare pentru indexul puzzle-ului și inițializarea tablei din ROM în loc de zero.

7.2. Afișarea numărului de mutări sau timer

Se poate adăuga un **contor de mutări** afișat pe SSD în timp ce jocul este în desfășurare:

- Un contor pe 8 biți (max 99 mutări) incrementat la fiecare apăsare de BTN_ENTER.
- Afișarea pe primele 2 cifre ale SSD a numărului de mutări (BCD encoding).
- Alternativ, un timer descrescător (ex: 5 minute) cu afișaj MM:SS pe 4 cifre SSD – la expirare → LOSE automat.

7.3. Evidențierea erorilor în timp real

Sistemul poate fi extins pentru a **detecta conflicte în timp real** (două valori identice pe aceeași linie/coloană/cadran):

- Adăugarea unui proces combinațional care verifică unicitatea per linie, coloană și cadran 2×2.
- Celulele în conflict pot fi afișate cu o culoare diferită (ex: roșu intens cu clipire) pentru feedback vizual imediat.

7.4. Niveluri de dificultate

Se pot adăuga **celule pre-completate** care nu pot fi modificate, creând puzzle-uri cu dificultate variabilă:

- Un semnal fix_mask (matrice booleană 4×4) blochează modificarea celulelor marcate ca fixe.
- Celulele fixe pot fi afișate cu un fundal diferit (ex: gri deschis) pentru a le diferenția de cele introduse de jucător.

7.5. Sunet – buzzer la câștig/eșec

- Placa Nexys 4 nu are buzzer integrat, dar un buzzer extern poate fi conectat la un pin GPIO.
- Un modulator PWM simplu poate genera tonuri diferite pentru WIN și LOSE.

Capitolul 8: Bibliografie

8. Digilent Inc., *Nexys 4 FPGA Board Reference Manual*, <https://digilent.com/reference/programmable-logic/nexys-4/reference-manual>, accesat 2025.
9. Xilinx Inc., *Vivado Design Suite User Guide*, UG910, 2023.
10. IEEE Standard VHDL Language Reference Manual, *IEEE Std 1076-2019*.
11. VESA, *VESA Monitor Timing Standard v1.0*, Video Electronics Standards Association, 1998.
12. Pong P. Chu, *FPGA Prototyping by VHDL Examples: Xilinx Spartan-3 Version*, Wiley-Interscience, 2008.
13. UTCN – Laborator Proiectarea Sistemelor Numerice, *Ghid/Îndrumător Proiecte*, 2024-2025.